The primary idea behind RiffMates is to create a space for classified ads, where bands can seek musicians and musicians can seek bands. You could manage all this through email, having the musicians send a note to your admins, but that is a lot of work for your admins. Instead, you want your musicians and others to be able to serve themselves.

Self-service sounds great, but what if two bands are rivals? You can't have the Sharks and the Jets editing each other's profiles—that won't end well. Multiuser sites really are about being able to control who does what where. To provide this kind of control, you need two things: authentication and authorization. These are related concepts, and one sometimes implies the other. *Authentication* is proving who you are, while *authorization* is the system allowing you to perform an action. User accounts pair a login ID and password to provide the authentication part, while your code determines authorization.

As part of authentication, Django comes with a built-in ORM Model object for tracking users and passwords. It handles the encryption of the passwords for you and validation when a user enters a password to prove who they are. This is accomplished through middleware. *Middleware* is code that sits between the routing of the URL and the calling of the view and can affect what is sent to the view or whether the view gets called at all. Django provides some middleware out of the box, including session management. *Session management* is what determines whether a user gets logged in or not and when to log them out automatically, expiring the session. As with most things in Django, middleware is pluggable, and you can write your own, but doing so isn't covered in this book. In figure 6.1, you can see the middleware component in the framework part of the diagram. Additionally, the view in the diagram has a decorator: `@login_required`. This is the simplest form of authorization: the view can only be used by those who are authenticated.

A view may also contain further authorization logic. For example, a page might be a private area only band members can see. The URL for such a page has a `Band` ID, is wrapped with the `@login_required` decorator to ensure a musician attempting access gets authenticated, and validates that the user account belongs to a musician that is a member of the band. If authorization fails, the usual response is to raise an `Http404` error.

As soon as you have user accounts, you have a password management problem. People are forgetful, and the late-night, hard-living lifestyle of musicians makes them even more so. Django provides views and utilities for both user creation and password management, but as most of this needs to look like your site, it isn't quite ready out of the box; you have to do some customization to make it work.

This chapter is all about user accounts: how to create them, how to manage them, how to use them to authenticate and authorize users, and how to manage password resets.